

Junming Liang

Tel: 86 18728445038 | Email: liangjunming_liang@126.com

Blog: <https://jasper-aa64.github.io>

EDUCATION

Anhui Normal University (AHNU)

Wuhu, China

- B.Eng. in Software Engineering*
- Average Score: 87.1/100
- Relevant Coursework:** Data Structures, Algorithm Design and Analysis, Operating Systems, Computer Networks, Database Principles, Object-Oriented Programming, Discrete Mathematics I&II, Probability and Mathematical Statistics

Sept 2023 – Jul 2027 (Expected)

AWARDS & HONORS

Silver Medal, ACM-CCPC 2025 China Collegiate Programming Contest (CCPC) Jinan Regional Contest Nov 2025

Bronze Medal, ACM-ICPC 2025 ICPC Asia Wuhan Regional Contest Nov 2025

First Prize, 22nd “May 1st” Mathematical Modeling Competition Jun 2025

First Prize, 11th AHNU “Challenge Cup” Extracurricular Academic and Technological Works Competition Mar 2025

Scholarships for Excellent Performance in Science and Technology Activities 2024, 2025, 2026

Second Prize, National College Student Mathematical Modeling Competition (Anhui Provincial) Dec 2024

PROFESSIONAL EXPERIENCE

C++ Engineer (Intern) | Nanjing Pushi Intelligent Technology Co., Ltd.

Apr 2026 – Sep 2026

- Developed and optimized low-latency C++ systems for quantitative trading on Linux, contributing to a factor-computation engine and trading services for TWAP position pushing, bidding strategies, and position management
- Profiled a performance-critical module and identified ~3.4% runtime overhead from kernel page-fault and page-zeroing paths; evaluated an allocator replacement via LD_PRELOAD without recompilation, reducing median wall-clock time by 4.77% with byte-identical factor outputs (permutation $p = 0.0312$)
- Optimized the TWAP position-push service for 5,000 instruments $\times$ 10 users, achieving zero dropped pushes, correctness PASS at 10,000- and 50,000-push verification tiers, and 1.2 ms p95 latency by moving database writes off the hot path, introducing per-session push queues, removing hot-path logging, and batching Redis operations
- Improved five bidding-strategy endpoints (add, edit, start, stop, delete) to the millisecond range, with maximum p95 latency of 45.7 ms, by asynchronously refreshing full-list Redis data and coalescing requests within short time windows
- Applied repeatable A/B and stress tests with correctness checks and recorded rollback decisions to validate performance changes, motivating my ongoing independent research on reliable evaluation of agent-generated optimizations

RESEARCH & PROJECTS

Research | Reliable Evaluation of Agent-Generated C++ Performance Optimizations

Jul 2026 – Present

GitHub: github.com/Jasper-aa64/promotion-gate-audit

- Built an agent-driven C++ optimization workflow separating patch generation from acceptance, integrating correctness checks, randomized paired measurements, window-level statistical inference, predefined budgets, and independent confirmation
- Evaluated an earlier acceptance gate on 30 strict-null patches screened to preserve executable tokens in a public C++ order-book program: 0 promotions versus 13 under a naive first-pair rule; follow-up simulations examined risk-power-cost trade-offs for seven sign-based procedures
- Evaluated an outcome-unknown optimization of a public C++ order-book system; withheld the merge recommendation after separate agent review exposed parsing and memory-safety defects, then coordinated agent-led repair and validation with 28 differential test cases, 1,375 byte-for-byte formatting comparisons, and ASan/UBSan checks
- Re-evaluated the corrected binary on the tested workload using 580 fresh confirmation pairs, measuring a 550.6 ms mean runtime reduction (95% CI: 527.4–573.7 ms), equivalent to a 30.0% reduction relative to the control mean; all 1,160 runs completed successfully with matching recorded program-state checks

Main Developer | AI Real-Time Voice Platform (Interview & IELTS)

Feb 2026 – Aug 2026

GitHub: github.com/Jasper-aa64/AI-realtime_speech-for-IELTS-Job_Interview

- Designed and built a C++17 real-time voice platform integrating ASR, TTS, VAD, and LLM services, supporting two applications on the same core: an AI technical-interview system and an IELTS speaking and writing trainer
- Structured the platform in four layers (Common, Service, Business, UI) decoupled by abstract interfaces, with a thread-safe global state machine as the sole coordination center across 8 transitions, and a 4-thread model (main, microphone, playback, receiving) coordinated by `std::atomic`, `std::mutex` and `std::condition_variable`
- Built the audio and transport path with cross-platform capture and playback at 16 kHz int16 in and 24 kHz float32 out, a 200 ms blocking frame balancing latency against CPU, and a Boost.Beast WSS persistent connection carrying a custom binary protocol (header/event/compression)
- Integrated an OpenAI-compatible libcurl client for structured-JSON LLM interactions, supporting resume-driven question generation, real-time scoring, and follow-up decisions; isolated Boost.Beast, PortAudio, and PoDoFo behind Pimpl, with spdlog, GTest, AddressSanitizer, and a Qt6 desktop GUI for JSON report export

- Developed the IELTS trainer around an interactive report where each sentence receives its own evaluation, edits can be made in place, accepted phrasings are added to a personal corpus, and revised text is resubmitted for re-scoring

Main Developer | **C++ MCP Server & Multi-Agent Coding System**

Mar 2026 – Apr 2026

- Implemented an MCP server in C++17 with pluggable dispatch for Tools, Resources, and Prompts, supporting dynamic registration and runtime routing over JSON-RPC transports including HTTP + SSE and stdio with Content-Length framing
- Extended the server with 13 C++-specific tools for file I/O, code execution, complexity analysis, and Git operations, with path-access restrictions, command allowlists, and quote-aware filtering of potentially dangerous command characters
- Applied fine-grained concurrency control with independent mutexes per module to avoid cross-module lock contention, and graceful shutdown via std::atomic and signal handling so in-flight requests complete and every resource is released on exit
- Orchestrated three agents (Coder, Reviewer, Tester) as a LangGraph state graph with up to three VERDICT-driven retries, role-based tool permissions, and both hosted LLM API and local Ollama backends; logged agent trajectories and exported them for offline analysis

Team Leader | **Summer Social Practice (App Development Program)**

Jul 2025 – Aug 2025

- Led the development of a full-stack WeChat mini-program for exhibition introduction and indoor navigation
- Managed task division, progress control, and official documentation, including requirements and API specs
- Successfully delivered and launched the program; obtained the National Software Copyright Certificate

HIGHLIGHTED TECHNICAL SKILLS

C++ & Performance:

- C++17/20, STL, RAII, templates, Pimpl, SIMD/AVX, cache optimization, memory pools, Lock-Free & Wait-Free Concurrency (SPSC/MPMC, Memory Ordering), NUMA, CPU affinity, RDTSC, perf, TMA, flame graphs.

Systems & Networking:

- Linux, IPC, epoll, mmap, thread pools, TCP/IP, HTTP, WebSocket, JSON-RPC 2.0, Content-Length framing, custom binary protocols

AI & Agent Engineering:

- MCP, LangGraph, LLM APIs, structured outputs, RAG, Ollama, ASR/TTS/VAD, AI coding agents

Tools:

- Python, CMake, vcpkg, Git, GTest, AddressSanitizer, Cppcheck, spdlog

ADDITIONAL INFORMATION

Languages: Chinese (Native), English (Proficient in academic & technical communication, IELTS: 6.5 (L: 7.5, R:7, W:6, S: 5.5))

Hobbies: Erhu (National Conservatory of Music Level 10), Chinese Painting